PitFusion3D

Pipeline optimization report

Raspberry Pi 5 target: RGB + thermal + Livox Avia fusion

6.29x

overall speedup across benchmarked stages

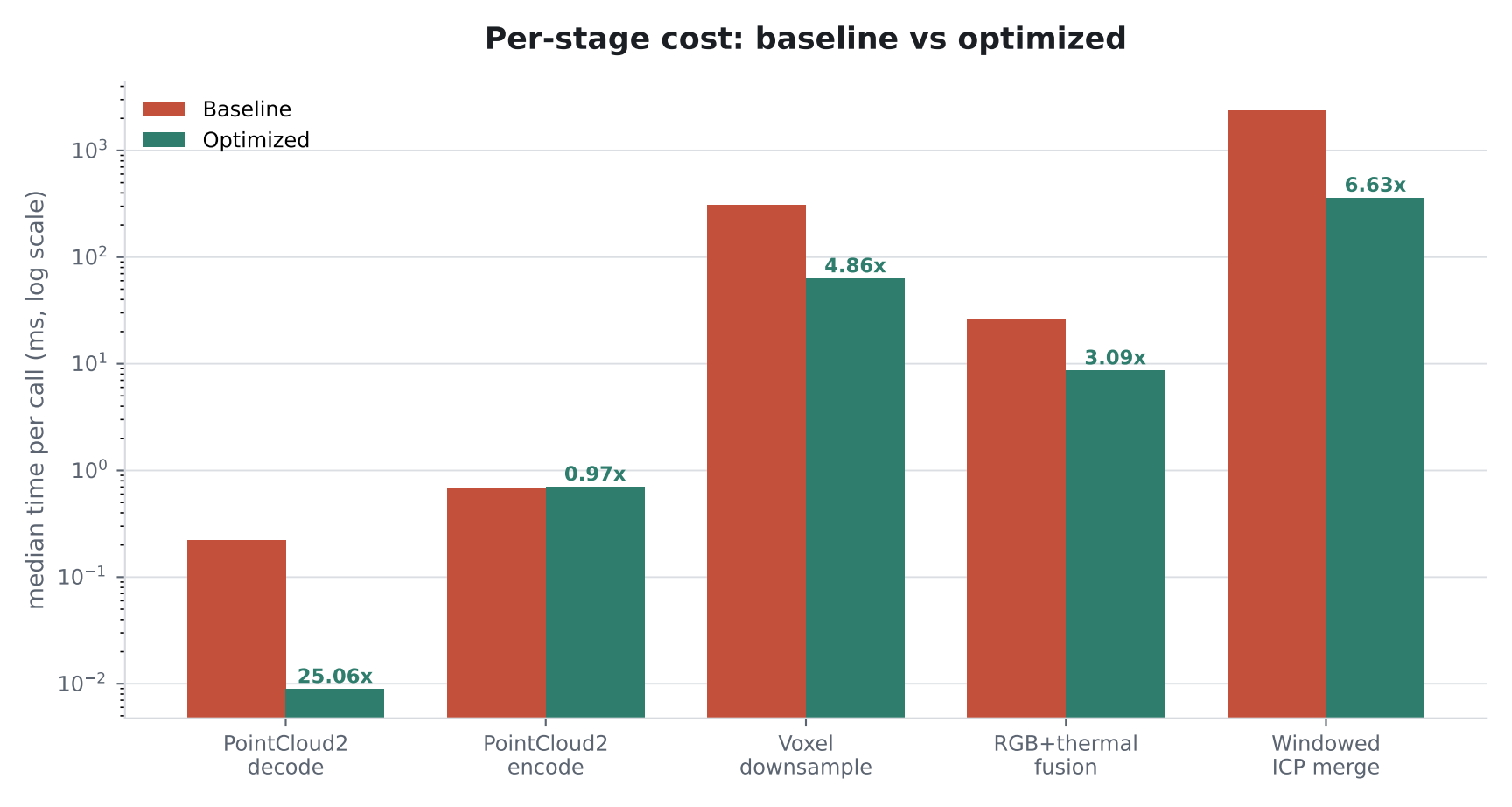
2707 ms -> 430 ms per merge window

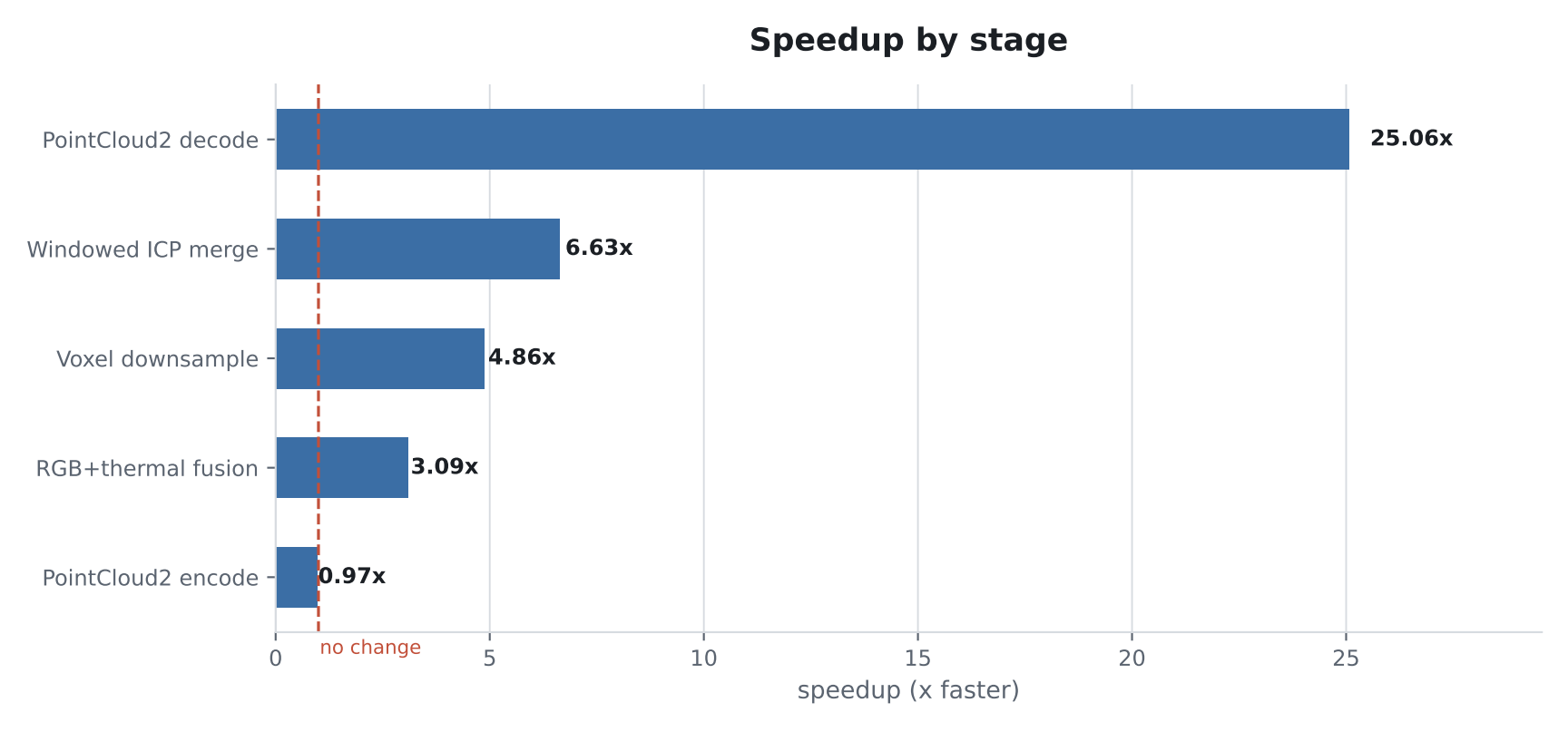
stage	baseline	optimized	speedup
PointCloud2 decode (24k pts, 32B stride)	0.22 ms	0.01 ms	**25.06x**
PointCloud2 encode (24k pts, 6 fields)	0.68 ms	0.70 ms	**0.97x**
Voxel downsample (120000 pts, 1cm)	304.50 ms	62.59 ms	**4.86x**
RGB+thermal fusion (24k pts, 1280x720)	26.54 ms	8.58 ms	**3.09x**
Windowed ICP merge (5 scans x 24k pts)	2375.23 ms	358.19 ms	**6.63x**

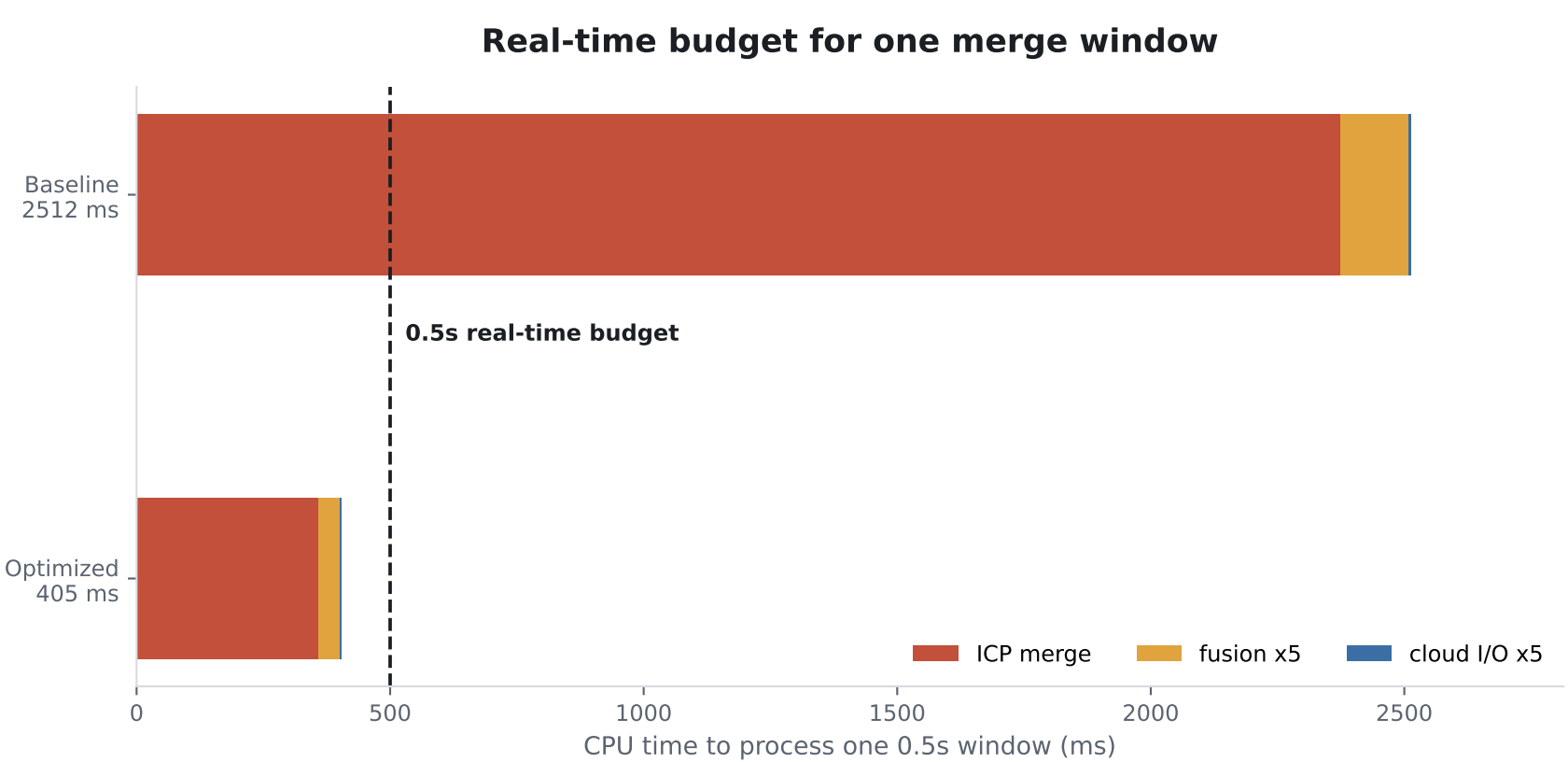
Correctness fix included in this release

The Livox resampler previously snapped every return to the nearest cell of the uniform gpu_lidar grid, quantising the scan onto only 120 distinct elevations and rendering as horizontal banding rather than the Avia rosette. Points are now rebuilt along each pattern ray's own azimuth/zenith using bilinear range interpolation.

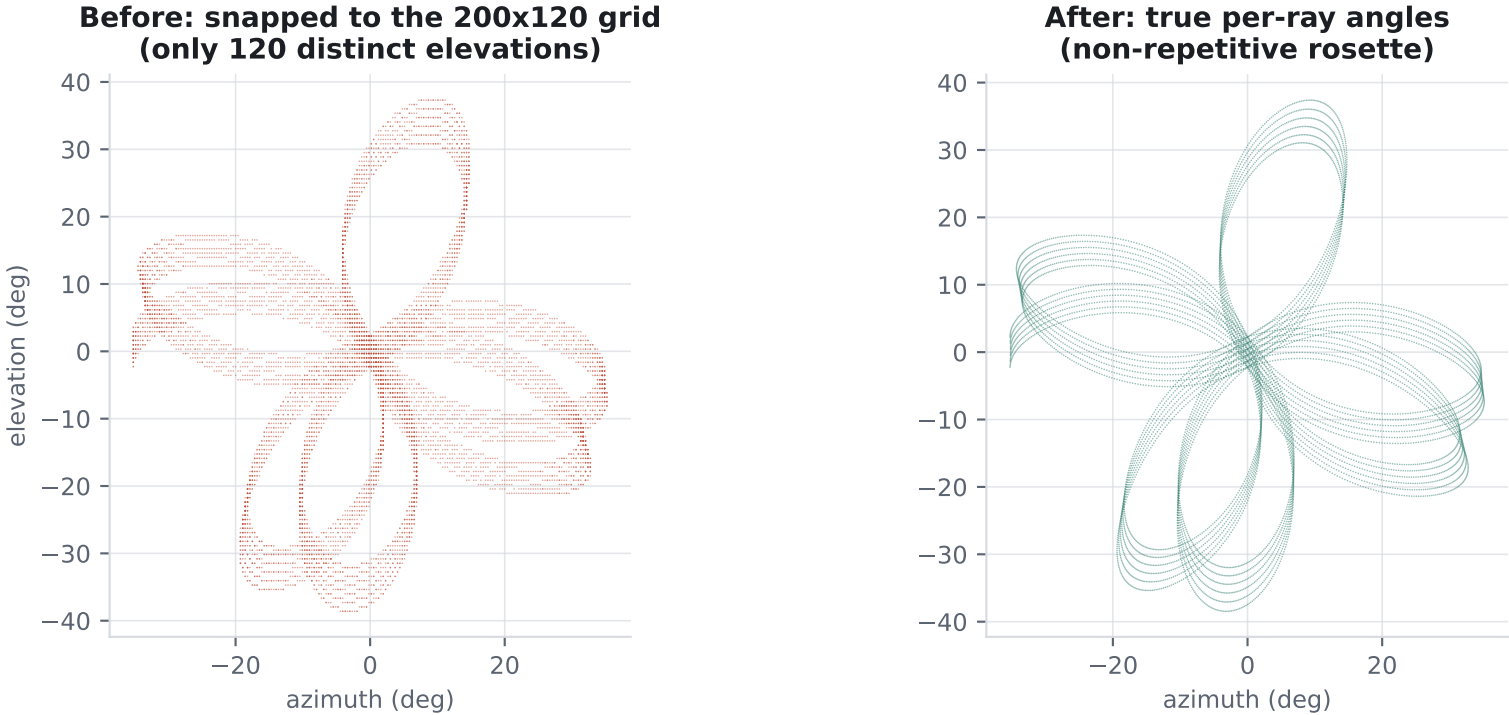
Measured on Linux-6.8.0-1065-raspi-aarch64-with-glibc2.39 | 4 cores | numpy 1.26.4


Per-stage cost: baseline vs optimized
Baseline
Optimized
median time per call (ms, log scale)
10^{3}
10^{2}
10^{1}
10^{0}
10^{-1}
10^{-2}
25.06x
0.97x
4.86x
3.09x
6.63x
PointCloud2 decode
PointCloud2 encode
Voxel downsample
RGB+thermal fusion
Windowed ICP merge


Speedup by stage
PointCloud2 decode
Windowed ICP merge
Voxel downsample
RGB+thermal fusion
PointCloud2 encode
25.06x
6.63x
4.86x
3.09x
0.97x
no change
0
5
10
15
20
25
speedup (x faster)


Real-time budget for one merge window
Baseline
2512 ms
Optimized
405 ms
0.5s real-time budget
ICP merge
fusion x5
cloud I/O x5
0
500
1000
1500
2000
2500
CPU time to process one 0.5s window (ms)

Livox Avia scan pattern reconstruction (9000 rays)



What changed, and why it was slow

PointCloud2 decode

Before sensor_msgs_py rebuilt an intermediate structured array per message.

After Map msg.data with a numpy dtype carrying the real field offsets and take the x/y/z/intensity block as one strided view - zero copies.

PointCloud2 encode

Before pc2.create_cloud re-packs the payload field by field in Python.

After Write the float32 matrix straight to msg.data with .tobytes(), since the output layout is already known and dense.

Voxel downsample

Before np.unique(keys, axis=0) lexsorts an (N,3) array - the dominant cost of every merge window.

After Hash the 3 integer voxel coordinates into one int64 key so the dedupe becomes a 1-D sort. Selection is bit-identical.

RGB + thermal fusion

Before Converted the entire 1280x720 thermal frame to Kelvin (921k pixels) when only ~24k points sample it, in float64, via cv_bridge.

After Gather the raw uint16 at the projected pixels first and convert only those ~24k values; stay in float32; fold the extrinsic hops into one (M, b) precomputed at startup; reuse the RGB projection when the rig is co-located.

Windowed ICP merge

Before Each scan rebuilt a KD-tree over an ever-growing raw map (O(N^2)), and the map was re-voxelised from scratch every iteration.

After Keep the running map voxel-downsampled and grow it incrementally, cap the ICP source/target sizes, and use distance_upper_bound so the tree prunes. Correspondence search is serial below 20k points because scipy's per-call thread spawn cost exceeded the parallel gain.